

PRAKTIKUM BASIS DATA

MODUL 11

No SQL Key Value Database



Oleh:

Davis Arvaputra Dwiansyah 103122400034

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL 11

1. Davviz adalah seorang Backend Engineer handal gg jago yang sedang membangun platform streaming musik generasi terbaru bernama Spotifine. Aplikasi ini akan memiliki jutaan pengguna aktif, sehingga Davviz membutuhkan database yang tahan banting, dan tidak akan mati meskipun salah satu servernya mengalami gangguan. Davviz ditugaskan untuk merancang database yang menyimpan Data Musisi di dalam platform Spotifine. Karena satu musisi bisa memiliki banyak genre dan pengikut dari berbagai negara, Davviz memilih Cassandra sebagai solusinya. Berikut atribut yang perlu dibuat :

- Artist ID: ID unik untuk setiap musisi (Primary Key).
- Artist Name: Nama lengkap musisi atau nama panggung.
- Followers: Jumlah pengikut (Tipe data int).
- Origin: Negara asal musisi.
- Genres: Daftar aliran musik yang diusung (Gunakan tipe data set karena satu musisi bisa memiliki genre lebih dari satu, misal: Pop dan R&B).

Jawaban:

Kolom	Tipe Data	Keterangan
artist_id	UUID	ID unik musisi (Primary Key)
artist_name	TEXT	Nama musisi
followers	INT	Jumlah pengikut
origin	TEXT	Negara asal
genres	SET<TEXT>	Daftar genre musik

2. Implementasi

a) Buat Keyspace

- Buat sebuah keyspace bernama spotifine_0801 yang akan digunakan sebagai tempat penyimpanan utama database aplikasi.
- Gunakan pengaturan replikasi SimpleStrategy dengan nilai replication_factor = 1.
- Tuliskan perintah query untuk membuat keyspace tersebut, lalu sertakan screenshot ketika keyspace berhasil dibuat dan saat dijalankan menggunakan perintah USE spotifine_0801.

b) Buat Table

- Buat tabel dengan nama musician_data di dalam keyspace spotifine_0801 sesuai atribut yang telah ditentukan sebelumnya oleh Davviz.
- Tuliskan query CREATE TABLE lengkap beserta atribut-atributnya dan lampirkan screenshot hasil pembuatan tabel.
- Tambahkan minimal 3 data musisi yang berbeda ke dalam tabel menggunakan perintah INSERT.
- Pastikan terdapat setidaknya satu musisi yang memiliki lebih dari satu genre pada

kolom bertipe SET.

- Tampilkan seluruh data yang sudah dimasukkan menggunakan query `SELECT * FROM musician_data;`
- Sertakan screenshot hasil akhir tabel yang telah berisi data musisi tersebut.

Jawaban:

a. Membuat Keyspace

```
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 2.2.3 | CQL spec 3.3.1 | Native protocol v4]
Use HELP for help.
WARNING: pyreadline dependency missing. Install to enable tab completion.
cqlsh> CREATE KEYSPACE spotifine_0801
... WITH replication = {
...     'class': 'SimpleStrategy',
...     'replication_factor': 1
... };
cqlsh> describe keyspace;
Not in any keyspace.
cqlsh> describe KEYSCAPE;

'keyscape' not found in keyspaces
cqlsh> describe keyspaces;

system_auth      "OpsCenter"          system_traces      praktikum
system            system_distributed  spotifine_0801

cqlsh> USE spotifine_0801;
cqlsh:spotifine_0801> |
```

b. Membuat Table Dan insert data Query di CassandraSQL

- Query Create Table :

```
cqlsh:spotifine_0801> CREATE TABLE musician_data (
...     artist_id UUID PRIMARY KEY,
...     artist_name TEXT,
...     followers INT,
...     origin TEXT,
...     genres SET<TEXT>
... );
cqlsh:spotifine_0801> describe musician_data;

CREATE TABLE spotifine_0801.musician_data (
  artist_id uuid PRIMARY KEY,
  artist_name text,
  followers int,
  genres set<text>,
  origin text
) WITH bloom_filter_fp_chance = 0.01
AND caching = '{"keys":"ALL", "rows_per_partition":"NONE"}'
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy'}
AND compression = {'sstable_compression': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND dclocal_read_repair_chance = 0.1
AND default_time_to_live = 0
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair_chance = 0.0
```

- Query Insert Value :

```
cqlsh:spotifine_0801> INSERT INTO musician_data (
...     artist_id,
...     artist_name,
...     followers,
...     origin,
...     genres
... )
... VALUES (
...     uuid(),
...     'The Weeknd',
...     120000000,
...     'Canada',
...     {'R&B', 'Pop'}
... );
cqlsh:spotifine_0801>
cqlsh:spotifine_0801> INSERT INTO musician_data (
...     artist_id,
...     artist_name,
...     followers,
...     origin,
...     genres
... )
... VALUES (
...     uuid(),
...     'Taylor Swift',
...     150000000,
...     'USA',
```

```

... )
... VALUES (
...     uuid(),
...     'Taylor Swift',
...     150000000,
...     'USA',
...     {'Pop', 'Country'}
... );
cqlsh:spotifine_0801>
cqlsh:spotifine_0801> INSERT INTO musician_data (
...     artist_id,
...     artist_name,
...     followers,
...     origin,
...     genres
... )
... VALUES (
...     uuid(),
...     'Rich Brian',
...     120000000,
...     'Indonesia',
...     {'Hip Hop'}
... );
cqlsh:spotifine_0801> |

```

- Output Dari insert Query dengan menggunakan select :

```

... );
cqlsh:spotifine_0801> SELECT * FROM musician_data;

```

artist_id	artist_name	followers	genres	origin
b946da2c-ad5d-4daf-83bd-4463be1fcf23	Taylor Swift	150000000	{'Country', 'Pop'}	USA
ef050cea-9ffe-409b-b68b-d3fa2b868f43	The Weeknd	120000000	{'Pop', 'R&B'}	Canada
eeca6b17-b859-443a-99a3-2564b22699da	Rich Brian	120000000	{'Hip Hop'}	Indonesia

```

(3 rows)
cqlsh:spotifine_0801> |

```

3. Apa keuntungan utama menggunakan tipe data set pada kolom genres di aplikasi Spotifine dibandingkan jika ia menggunakan tabel relasional (RDBMS) biasa?

Jawaban :

Penggunaan tipe data *SET<TEXT>* pada kolom *genres* di Spotifine lebih menguntungkan karena satu data lagu atau musisi dapat menyimpan beberapa genre sekaligus dalam satu kolom, misalnya {'Jazz', 'Blues', 'Soul'}, tanpa harus membuat tabel relasi tambahan seperti pada RDBMS. Cara ini membuat struktur database lebih praktis dan proses pengambilan data menjadi lebih cepat karena tidak memerlukan banyak *join*. Selain itu, tipe *set* juga secara otomatis mencegah data genre yang sama tersimpan lebih dari sekali, sehingga data menjadi lebih konsisten, hemat penyimpanan, dan sesuai untuk aplikasi musik berskala besar yang membutuhkan performa tinggi.